Hospital Resource Management System

P. Uday Kumar & Hari Charan

Made with GAMMA

OVERVIEW

Project Framework & Data Structure

Managing hospital records manually is inefficient, time-consuming, and prone to errors. This system uses a linked list-based approach to dynamically store, update, retrieve, and delete patient information without memory limitations — implementing full CRUD operations in a menu-driven C program.

Patient Record Node

Each node stores Patient ID, Name, Age, Disease, and a pointer to the next node in the list.

Singly Linked List

Dynamic size with no fixed limit like arrays. Memory is allocated at runtime using `malloc()`.

Dynamic Memory

Efficient insertion and deletion with memory allocated and freed at runtime — no static size constraints.

Menu-Driven Interface

A user-friendly interactive console menu guides the user through all CRUD operations step by step.

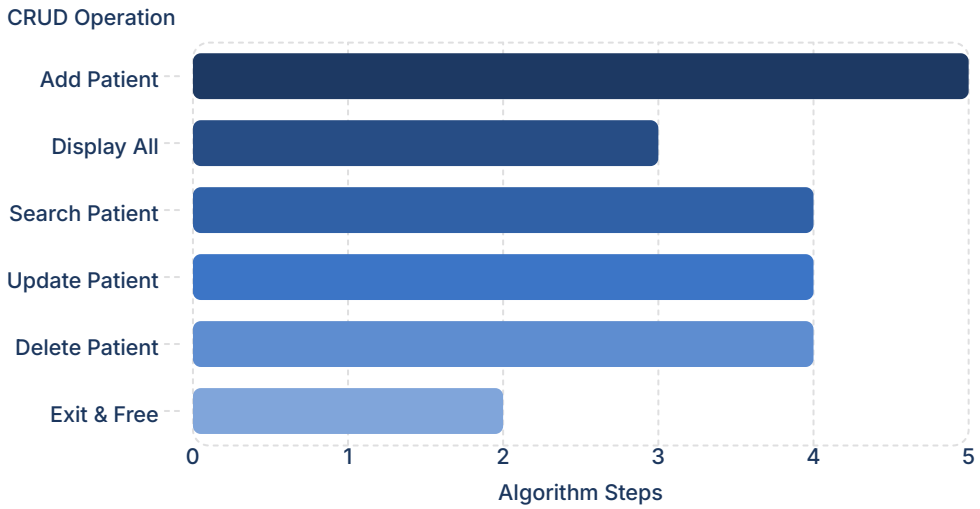
Made with GAMMA

ALGORITHM

CRUD Operation Algorithms

Each operation in the system follows a clear algorithmic pattern to ensure correctness, memory safety, and data integrity throughout the linked list.

- **Add Patient (Create):** Allocate memory via malloc(), read patient details, create a new node, insert at end of list, and update links.
- **Display Patients (Read):** Start from head, traverse until NULL, and print each node's data sequentially.
- **Search Patient:** Input Patient ID, traverse list, display record if ID matches, else show "not found".
- **Update Patient:** Input Patient ID, traverse list, update details if found, else show "not found".
- **Delete Patient:** Input Patient ID, traverse with previous pointer, adjust links to remove node, then free memory using free().



Made with GAMMA

COMPILATION

Compilation & Execution Guide

The program is compiled and run using the GCC compiler. Follow the steps below to get the system up and running from source code to an executable in minutes.

1 Step 1: Save the Source File

Save the C source code as `hospital.c` in your working directory on your machine.

2 Step 2: Open Terminal

Open a terminal or command prompt and navigate to the directory where `hospital.c` is saved.

3 Step 3: Compile with GCC

Run the compile command: `gcc hospital.c -o hospital` to generate the executable output file.

4 Step 4: Run the Executable

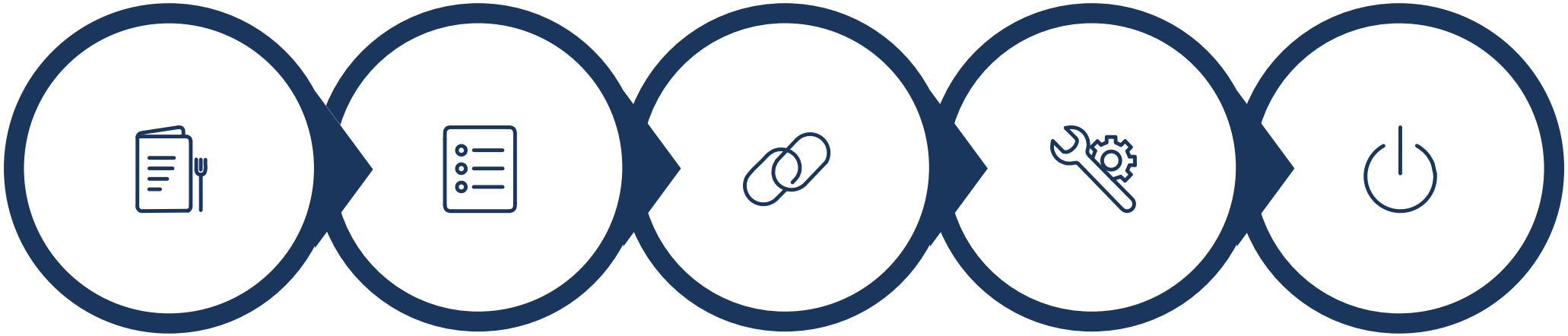
Execute the program by running `./hospital` in the terminal. The menu-driven interface will launch immediately.

Made with GAMMA

PROCESS

CRUD Operation Flow

The system follows a structured menu-driven flow, guiding the user from program launch through each CRUD operation to clean program exit with full memory deallocation.



Menu | Select CRUD | Traverse List | Perform Op | Free & Exit

Each operation traverses the singly linked list from the head node, performs the required action on the matching Patient ID, and safely updates pointers — ensuring data integrity and no memory leaks throughout the session.

Made with GAMMA

IMPACT

Key System Highlights

The Hospital Resource Management System addresses the core inefficiencies of manual record-keeping. The highlights below demonstrate the practical value of using dynamic memory allocation and linked lists for real-world hospital data management.

5

CRUD Operations

Add, Display, Search, Update, and Delete — five fully implemented operations covering every aspect of patient record management.

0

Fixed Memory Limit

Unlike static arrays, the linked list imposes no fixed memory limit — records grow and shrink dynamically at runtime using `malloc()` and `free()`.

4

Data Fields / Node

Each patient node stores four fields — Patient ID, Name, Age, and Disease — alongside a pointer to the next record.

Made with GAMMA

FEATURES

System Features & Best Practices

The following features make this system efficient, extensible, and educational. These design choices reflect best practices in C programming and data structure implementation for real-world applications.

Menu-Driven Interface

An intuitive numbered menu presents all operations clearly, making the system easy to use without prior technical knowledge.

Dynamic Memory Allocation

Memory is allocated on demand using `malloc()` and released safely using `free()`, eliminating wasted static memory.

Efficient Linked List Handling

Insertion and deletion are performed efficiently by updating node pointers, avoiding costly array shifting operations.

Patient Search & Update

Patient records can be located by ID and updated in-place, ensuring the dataset remains current and accurate at all times.

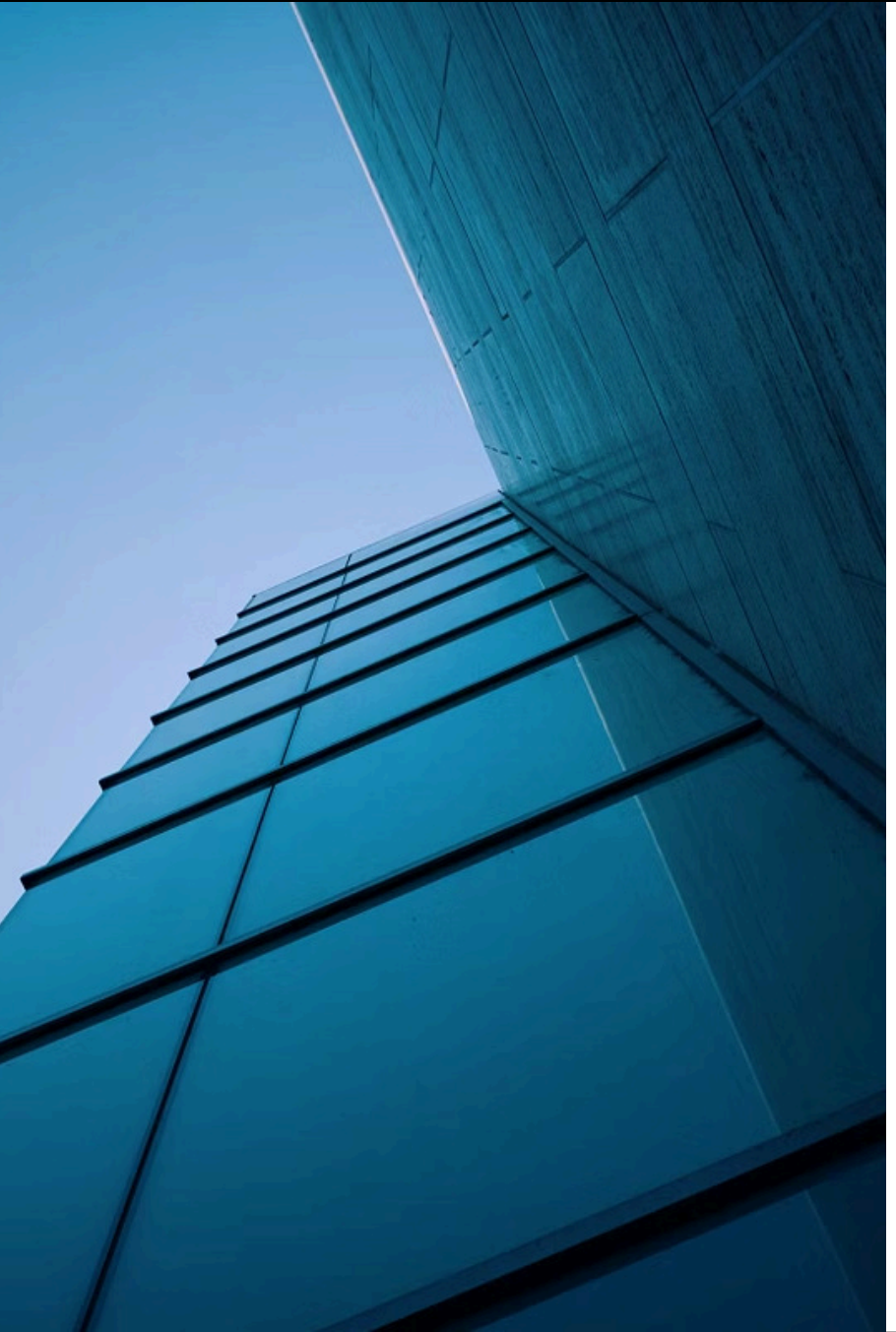
Safe Deletion with Free

Deleted nodes have their memory freed immediately, preventing memory leaks and maintaining system performance over long sessions.

Scalable Foundation

The architecture provides a strong foundation for real-world applications such as hospital databases and larger health management systems.

Made with GAMMA



CONCLUSION

Project Summary & Outcomes

The Hospital Resource Management System successfully demonstrates the use of linked lists and dynamic memory allocation in C. It efficiently performs all CRUD operations and overcomes the limitations of static data structures, providing a strong foundation for real-world healthcare applications.

Language & Implementation

Built entirely in C using `malloc()` and `free()` for dynamic memory management, with a singly linked list as the core data structure for all patient records.

Full CRUD Coverage

All five operations — Add, Display, Search, Update, and Delete — are fully implemented and accessible through the menu-driven interface, covering every record management scenario.

Real-World Applicability

This project bridges academic data structure concepts with practical healthcare needs, forming a scalable base for more advanced hospital information and database management systems.

Files used: `hospital.c` — the complete source code containing all CRUD logic, linked list operations, and menu interface.

Made with GAMMA